

UCSB's International Capture The Flag Competition 2010 Challenge 6: Fear The EAR

UCSB's International Capture The Flag Competition 2010 Challenge 6: Fear The EAR - Author not stated, bryceboe.com.

- Published: date not stated
- Original: <<https://bryceboe.com/2010/12/09/ucsbs-international-capture-the-flag-competition-2010-challenge-6-fear-the-ear/>>
- Preserved from: <https://bryceboe.com/2010/12/09/ucsbs-international-capture-the-flag-competition-2010-challenge-6-fear-the-ear/> (live) on 2026-08-08
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Each year the Security Lab at UCSB hosts the [International Capture the Flag](#) competition, an approximately eight-hour security competition pitting [security groups at various universities](#) around the world against each other. Last year I had the privilege of [contributing significantly](#) to the setup on the iCTF, and later publishing and presenting a paper, "[Organizing Large Scale Hacking Competitions](#)" at [DIMVA 2010](#). This year, I finally had an opportunity to take Giovanni Vigna's security course, thus allowing me to participate in the competition. I led the team, "Tr0llF4ce Pwns You", and we did decently well considering this was the first hacking competition for many on our team.

Despite my participation, I was still able to contribute a challenge to the competition, which ended up being the 800 point challenge 6 in the competition. The primary reason for the challenge was to draw attention to a previously unpublicized web vulnerability that we are calling the **Execution After Redirect**, or **EAR Vulnerability**. This vulnerability is exactly as the name states, however to be a bit more precise our exact definition of the vulnerability is, "code that executes after the developer's intended termination point". The developer's intended termination point is often indicated by a server-initiated redirect, or more precisely an HTTP [301](#), [302](#), [303](#), or [307](#) status code along with an HTTP Location header. It is important to note that there are cases in which the developer intends server side code to continue executing following a redirect. Such cases are not EAR vulnerabilities.

[Adam Doupé](#) and I are in the process of writing a paper describing the complete details of the EAR vulnerability. For the purpose of understanding this challenge, you need only know that when a modern browser receives the redirect, any data sent in conjunction with the server-initiated

redirect will not be displayed as the browser automatically fetches the resource indicated by the redirect. Furthermore, tools such as wget, curl and python's urllib, among others, also automatically handle the redirect in their default configuration thus making the EAR vulnerability that much more difficult to detect.

With that said, the following is a complete walkthrough of solving my iCTF 2010 challenge:

Teams were presented with the message, "Obey the error messages and find the secret at <http://10.15.3.1:8000>." Upon visiting the URL teams would see the title, "User Administration" [this video](#) playing immediately, and a simple file upload submission form. The video was mostly an annoying red herring, however, some of the keywords in the video happened to be valid user names that could be beneficial. The submission form contained an upload field called *control* and a hidden field called *csrf* whose value was a randomly generated integer valid for 30 seconds that could only be used once to indicate a valid form

The first real part of the challenge required discovering the control file format. This part was not meant to be difficult, thus the server intentionally leaked a plethora of information through error messages that were delivered to the user via a cookie along with a server-initiated redirect to the same page. When a GET request was issued containing the error message cookie, the error message was simply displayed to the user on the page. While the code to handle the error message did contain an [XSS vulnerability](#), it was irrelevant to the challenge. Sending error messages through cookies followed by redirects was my hint to teams to investigate what else was sent in the first server response.

The specific error messages allowed teams to quickly discover the file had to be three lines and less than 40 characters where the first line was for the username, and the second for the password. At this point, teams had to guess usernames and passwords, however this process was pretty trivial due to error messages indicating that usernames could be at most 6 characters a-z, and the password could be at most two numerical digits. The passwords could be easily brute forced, however the username space was 276-1 in size which is not feasible to brute force. Thus, I populated the database with many guessable usernames such as, 'user', 'ictf', 'a', 'admin', 'root', 'dev', 'test', as well as some keywords from the annoying video. Further error messages indicated invalid usernames, incorrect passwords, inactive accounts, and non-administrator accounts allowing teams to guess valid username and password combinations in a minimal amount of time.

The service contained two types of users: *administrators*, and *regular* users. Both types of users could be either in an *active* or *inactive* state. In this challenge, all guessable administrator accounts were inactive, and all regular users were active. The second part of the challenge was to discover the Execution After Redirect Vulnerability. More specifically, teams needed to discover that upon logging in with a regular user account the administrator console view was sent in the response along with the redirect and cookie containing the error message stating that regular users could not send commands. As previously mentioned, the detection of the EAR vulnerability is not trivial thus this process required teams to do something special in order to view the raw response.

Once the vulnerability was discovered, teams learned, via an error message, that they could send the command "help" to list all the administrator commands, thus informing them about the other commands, "add", "info", and "list". The add command simply exposed the information that the SQLITE database was read only, the info command listed the info field for the specified user, and

the list command listed all the active users in the system along with their info field. The teams could use the info command to discover the special admin account that allowed them to successfully view the administrator console without exploiting the EAR vulnerability. The list command also told the teams that there were five disabled users.

The third and final part of the challenge was to discover and exploit the [SQL injection vulnerability](#) in the info command. Two caveats of this part were that the SQL injection vulnerability had to be performed without standard whitespace, and the teams had to get the info field for the user secret. Neither of these proved difficult for the teams that made it this far.

During the competition, 69 of the 72 teams attempted to solve my challenge, of which 44 teams were able to submit valid control files. 34 of those teams, successfully guessed regular user accounts, thus exposing them to the Execution After Redirect Vulnerability. However, as indicated by the teams who successfully ran the “help” command, of these 34 teams, only 12 of them discovered the EAR vulnerability. Finally of those 12 teams, 7 successfully exploited the SQL injection vulnerability that provided them with: The secret is: <http://hackiswack.com/videos/viewvideo/23/hack-is-wack/blocka-blocka>

The 8 successful control files sent over the course of the competition were:

```
a\n50\ninfo 'OR(1)LIMIT/**/5,1--\n
zzyzx\n83\ninfo '/**/OR/**/pass<83;/**\n
a\n50\ninfo 0'/**/or/**/pass='66\n
zzyzx\n83\ninfo root'OR'1'='1'LIMIT'5','1\n
user\n50\ninfo ab'or/**/oid=6;--\n
a\n50\ninfo '/**/or/**/oid=6;--\n
zzyzx\n83\ninfo '/**/or/**/user/**/like's%\n
zzyzx\n83\ninfo 'OR(user='secret')--\n
```

My own control file used in testing:

```
a\n50\ninfo z'or"user"='secret
```

I think the fact that just over one third of the teams in our hacking competition discovered this vulnerability shows how dangerous it can be, and hence the title, Fear the EAR.

```
conn = sqlite3.connect(DATABASE)
try:
    c = conn.execute('select * from users where user=?', (user,))
    result = c.fetchone()
if not result:
    return self.send_redirect(S['nonexist'] % user)
_, user_pswd, info, is_admin, is_active = result
if pswd != user_pswd:
    return self.send_redirect(S['mismatch'] % user)
if not is_active:
    return self.send_redirect(S['inactive'] % user)
if not is_admin:
    self.send_redirect(S['admin'] % user)
self.process_command(conn, user, *cmd.split(' '))
finally:
    conn.close()
```

You can find the complete source code for this challenge [here](#). It simply requires running a single python file. The bug in the code that produces the vulnerability occurs on line 413 (line 13 above) in which I neglected to return from the function when I called `send_redirect` as in all other instances.

If you worked on this challenge in the competition let me know what you thought. As always I appreciate all feedback, and challenge related feedback will help me create better challenges for next year's iCTF.